HARRY WIK & VICTOR PEKKARI

GRAPH NEURAL NETWORKS

GRAPH NEURAL NETWORKS

HARRY WIK & VICTOR PEKKARI

Training GNNs on Large Graphs via Graph Databases

February 2026 –

Harry Wik & Victor Pekkari: *Graph Neural Networks,* Training GNNs on Large Graphs via Graph Databases, © February 2026

Ohana means family.
Family means nobody gets left behind, or forgotten.

— Lilo & Stitch

Dedicated to the loving memory of Rudolf Miede.

1939 – 2005

ABSTRACT

Short summary of the contents in English. . . a great guide by Kent Beck how to write good abstracts can be found here:

https://plg.uwaterloo.ca/~migod/research/beckOOPSLA.html

ZUSAMMENFASSUNG

Kurze Zusammenfassung des Inhaltes in deutscher Sprache. . .

PUBLICATIONS

This is just an early – and currently ugly – test!

This might come in handy for PhD theses: some ideas and figures have appeared previously in the following publications:

[1] Tobias Isenberg, André Miede, and Sheelagh Carpendale. "A Buffer Framework for Supporting Responsive Interaction in Information Visualization Interfaces." In: *Proceedings of the Fourth International Conference on Creating, Connecting, and Collaborating through Computing (C^5 2006)*. IEEE, 2006, pp. 262–269. ISBN: 978-0-7695-2563-1.

[2] Ulrich Lampe, Markus Kieselmann, André Miede, Sebastian Zöller, and Ralf Steinmetz. "A Tale of Millis and Nanos: On the Accuracy of Time Measurements in Virtual Machines." In: *Proceedings of the Second European Conference on Service-Oriented and Cloud Computing (ESOCC 2013)*. Springer, 2013, pp. 172–179. ISBN: 978-3-642-40650-8.

[3] Ulrich Lampe, Qiong Wu, Ronny Hans, André Miede, and Ralf Steinmetz. "To Frag Or To Be Fragged – An Empirical Assessment of Latency in Cloud Gaming." In: *Proceedings of the Third International Conference on Cloud Computing and Services Science (CLOSER 2013)*. 2013, pp. 5–12. ISBN: 978-898-8565-52-5.

[4] André Miede. "Theses and other Beautiful Documents with `classicthesis`." In: *TUGboat – The Communications of the TEX Users Group* 31.1 (2010), pp. 18–20. ISSN: 0896-3207.

[5] André Miede, Gökhan Şimşek, Stefan Schulte, Daniel F. Abawi, Julian Eckert, and Ralf Steinmetz. "Revealing Business Relationships – Eavesdropping Cross-organizational Collaboration in the Internet of Services." In: *Proceedings of the Tenth International Conference Wirtschaftsinformatik (WI 2011)*. Vol. 2. 2011, pp. 1083–1092. ISBN: 978-1-4467-9236-0.

[6] Hsin-Yi Tsai, Melanie Siebenhaar, André Miede, Yu-Lun Huang, and Ralf Steinmetz. "Threat as a Service? Virtualization's Impact on Cloud Security." In: *IEEE IT Professional* 14.1 (2012), pp. 32–37. ISSN: 1520-9202.

Attention: This requires a separate run of `bibtex` for your `refsection`, e. g., `ClassicThesis1-blx` for this file. You might also use `biber` as the backend for `biblatex`. See also http://tex.stackexchange.com/questions/128196/problem-with-refsection.

ACKNOWLEDGMENTS

CONTENTS

1 INTRODUCTION 1
1.1 Research Questions 1
1.2 Scope and Limitations 2

2 BACKGROUND 3
2.1 Traditional Graph Representation Learning 3
2.1.1 Transductive Nature of Traditional GRL 4
2.2 Neural Networks 5
2.2.1 Weight Optimization 5
2.3 Graph Neural Networks 6
2.3.1 Graph Neural Network Architecture 7
2.4 Graph Neural Networks and Large Graphs 7
2.4.1 Training on Large Datasets Under Memory Constraints 8
2.4.2 Subgraph Sampling 9
2.5 PyTorch Geometric 10
2.6 Neo4j 10

3 METHODOLOGY 11

A APPENDIX 13

BIBLIOGRAPHY 15

LIST OF FIGURES

LIST OF TABLES

LISTINGS

ACRONYMS

1 INTRODUCTION

Graph structures can be helpful as a tool in modeling several important domains. Examples of applications of such models are found in domains as different as that of predicting likely chemical bonds between atoms within medical science, and the problem of fraud detection in the field of finance. The prevalence of graph-structured data across many domains creates a need for efficient and accurate methods for modeling and analyzing graphs.

GNNs have emerged as a leading approach for representation learning on graphs, combining the structural intuition behind classical graph algorithms with the expressiveness and inductivity of neural networks. Unlike traditional graph representation learning (GRL) algorithms, GNNs are inductive in nature, and unlike most traditional neural networks they do not treat every input independently of other inputs. This allows GNNs to capture graph structure as well as generalize to unseen nodes, making them highly effective when analyzing graph-structured data. However, the graph structure also introduces a new challenge absent from traditional neural networks: the challenge of growing training data. The inter-connectivity between datapoints in a graph makes it impossible to consider datapoints independently, as is typically assumed in stochastic gradient descent (SGD).

A natural solution is to store the graph in a dedicated graph database and query it during training for the relevant neighbors of nodes in a mini-batch. Graph databases are especially built for efficient graph traversal and feature look-up—precisely the operations needed when training GNNs on large graphs. Yet there have been few studies exploring synergies between GNNs and graph databases.

This thesis explores whether graph databases can be used as a scalable and efficient solution for training GNNs on large graphs. We build upon the `PyTorch Geometric` machine learning library and couple it with subgraph fetching from `Neo4j`, where the graph data resides.

1.1 RESEARCH QUESTIONS

This thesis addresses the following research questions:

1. **RQ1 — Bottlenecks:** What are the primary bottlenecks in GNN training when using a graph database for graph storage, and to what extent can they be mitigated?

2. RQ2 — **Sampling:** Which subgraph sampling strategies are most efficient when the graph is stored in a graph database?

3. RQ3 — **Graph size:** For which graph sizes does a graph-database-based storage approach provide the greatest benefits?

1.2 SCOPE AND LIMITATIONS

The experiments in this thesis are conducted on four datasets of varying sizes using a two-layer Graph Convolutional Network (GCN) for node classification. The main emphasis is on the training pipeline, rather than on a particular downstream task or model architecture. Since the core stages of the training pipeline remain broadly similar across multiple graph learning tasks, the conclusions may have relevance beyond the specific task considered here.

2 BACKGROUND

Graphs in mathematics are used to model pairwise relationships between objects. Objects are modeled as nodes $\mathcal{V}$, and their relations are called edges $\mathcal{E}$. Many domains benefit from modeling data as objects (nodes) and relationships (edges). Consider, for example, social networks, protein–drug interactions, or railway networks. Studying graph structure and node features allows us to predict properties of the graph. In fraud detection, for instance, nodes can represent accounts and edges transactions. By modeling typical graph evolution—such as which transactions and accounts are added—it becomes possible to detect anomalous transactions that deviate from expected patterns.

When making predictions about graphs it is often beneficial to first create an embedding of the graph. The optimal embedding depends on the downstream task, and there are several ways to produce one: transductively with traditional GRL approaches, or inductively with GNNs.

This chapter provides a fundamental understanding of traditional GRL algorithms and neural networks, and describes how GNNs combine ideas from both. From traditional GRL they inherit the idea that node embeddings should reflect graph structure, while from neural networks they adopt a learnable, parameterized function. We also discuss challenges that arise when working with GNNs on large datasets—challenges that do not exist in traditional neural networks. Finally, we give a brief introduction to `PyTorch Geometric`, the machine learning library used during experiments, and to `Neo4j`, the graph database used as graph storage.

2.1 TRADITIONAL GRAPH REPRESENTATION LEARNING

In graph representation learning we aim to find a representation of the graph that encodes its structure. Such embeddings are typically used in downstream tasks such as node classification, link prediction, or clustering. Finding a good embedding is somewhat dependent on the downstream task.

The simplest way of constructing node embeddings is to manually select graph statistics as features. Common examples include node degree, clustering coefficient, and centrality measures such as eigenvector centrality or PageRank, which aim to capture local and global importance.

A more advanced class of traditional GRL methods learns shallow node embeddings by capturing similarities between nodes. Some of these methods can be formulated as matrix factorization approaches, where the goal is to approximate a node–node similarity matrix, often derived from the adjacency matrix or its powers. A key limitation of manually designed statistics is that they are not learned but instead depend on what the user believes is important for the task, making them inflexible and limited in expressive power.

A third type of traditional node embedding is the random-walk-based algorithm. These methods work by sampling random sequences of nodes and then learn embeddings by maximizing co-occurrence between nodes that appear within the same random walk. Such methods have also been shown to perform a form of matrix factorization over node co-occurrence derived from random walks.

The goal is to learn a decoding function that gives the probability of visiting node v on a random walk of length T starting from node u:

$$\mathbf{DEC}(\mathbf{z}_u, \mathbf{z}_v) \triangleq \frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{v_k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}} \approx p_{\mathcal{G},T}(v \mid u).$$

2.1.1 *Transductive Nature of Traditional GRL*

A common feature of all GRL algorithms discussed above—whether using node statistics, matrix factorization, or random walks—is that no weights are shared between nodes: no embedding-generating function is learned, only node embeddings for the current nodes in the graph. If new nodes are introduced, not only is there no way to compute their embeddings, the embeddings of the existing nodes also become inaccurate (the latter also occurs if nodes are removed).

Consider a simplified version of the optimization problem for random walk algorithms:

$$\min_{\{z_x \,:\, x \in \mathcal{V}\}} \sum_{(u,v) \in \mathcal{D}} -\log\Big(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)\Big),$$

where $\mathcal{D}$ is the set of node pairs encountered on random walks, and $\mathcal{V}$ is the set of nodes in the graph. The optimization problem above optimizes the actual node embeddings directly, rather than some embedding-generating function. This is what makes all traditional representation learning algorithms *transductive* in nature.

Consider a social network in which people are nodes and their relationships are edges. If a transductive graph representation is used, we learn feature vectors for specific people in the graph. If more people are introduced to the network, we must recompute every person's feature vector.

2.2 NEURAL NETWORKS

In contrast to the transductive nature of traditional GRL, neural networks are *inductive*: they learn a generalizable function.

Neural networks are models composed of interconnected processing units called neurons. How neurons are connected depends on the model architecture. Some neurons receive external input and pass it further into the network; others aggregate information from preceding neurons; and output neurons produce the final result. Every connection between two neurons is associated with a weight, by which the activation of the upstream neuron is scaled before reaching the next neuron. The network learns patterns by adjusting these weights.

The neural network can be defined as a function of weights θ that takes an arbitrary input x and outputs $\hat{y}$:

$$f(x, \theta) = \hat{y}.$$

The network learns by optimizing θ to minimize a loss function $\mathcal{L}$ over training data X, which quantifies the prediction error—either supervised (given a desired output y) or unsupervised (without a ground-truth outcome):

$$\min_{\theta} \mathcal{L}(X, \theta) = \sum_{x \in X} l\,(\,f(x, \theta)\,,\,y\,). \qquad \text{(supervised)}$$

The training data consists of input datapoints $\{x \in X_{\text{train}}\}$ mapped to targets $\{y \in Y_{\text{train}}\}$, forming the training set $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$. The idea is to maximize the likelihood $p(y_i \mid x_i, \theta)$ of the network producing the correct output for each input by adjusting the weights, in the hope that the learned weights generalize to unseen data. The model's generalization ability is evaluated on a separate test set drawn from the same distribution as the training data.

2.2.1 *Weight Optimization*

The optimization method used to find the optimal weights is typically some variant of gradient descent (GD). GD involves iteratively updating the model weights based on the current gradient of the loss function. Differentiating the loss with respect to the weights $\nabla_\theta \mathcal{L}$ gives the direction along which the loss increases most steeply; hence the negative gradient indicates the direction of steepest descent.[1] The weights are updated in proportion to the gradient with step size η, commonly called the learning rate:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla_{\theta^{(k)}} \mathcal{L}.$$

1 GD converges to the global minimizer for convex differentiable functions with appropriate step sizes. The loss function for neural networks is generally not convex, but variants of gradient descent are still used to find "good" weights.

Datapoints are typically assumed to be independent in standard supervised learning, meaning each prediction $\hat{y}_i$ depends only on x_i. Under this assumption the full gradient is a sum over all datapoints, making vanilla gradient descent computationally expensive. Stochastic gradient descent (SGD) approximates this gradient using a uniformly sampled mini-batch, reducing computation per update. Although SGD does not guarantee a decrease in loss at every step, its expected gradient is an unbiased estimate of the full-batch gradient, yielding convergence in expectation under the independence assumption.

This approach also alleviates memory constraints. Full-batch gradient descent requires access to the entire training set for each update, which may not fit in memory. By computing gradients on mini-batches, SGD limits memory usage and allows the batch size to be tuned to the available hardware.

2.3 GRAPH NEURAL NETWORKS

Graph neural networks incorporate ideas from traditional graph representation learning by preserving the intuition that node representations should reflect the graph structure—i. e., nodes that are close or structurally similar should have similar embeddings. However, instead of relying on fixed similarity measures or directly optimizing node embeddings, GNNs use a neural network architecture to learn this relationship.

By combining this structural inductive bias with a parameterized, trainable function, GNNs learn to generate node embeddings through neighborhood aggregation. This allows the model to adapt representations to the downstream task via gradient-based optimization, rather than relying on manually designed features or predefined objectives. As a result, GNNs are more expressive and flexible than traditional GRL methods. At the same time, they have a fundamentally different architecture: instead of learning embeddings tied to specific nodes in a fixed graph, they learn a shared function that generates embeddings for any node based on its features and local neighborhood.

Returning to the social network example from Section 2.1: when a new person joins the network, an embedding can be generated directly using the learned embedding function. Unlike transductive methods, there is no need to retrain the model or recompute embeddings for existing nodes. The same function can also be applied to an entirely new social network, as long as the underlying data distribution is similar to that seen during training. This ability to generalize to unseen nodes and graphs is what makes GNNs *inductive*.

2.3.1 *Graph Neural Network Architecture*

The assumption of independence between datapoints is fundamentally violated in graph-structured data. In a graph, each node is explicitly defined through its relationships to other nodes; considering nodes in isolation therefore discards essential information. The label or representation of a node is often highly dependent on its neighborhood structure and on the attributes of nearby nodes.

GNNs address this limitation by explicitly modeling relational dependencies. Instead of learning a function where the output for a node depends solely on its own features,

$$f(x_i, W) = \hat{y}_i,$$

GNNs learn representations that incorporate information from a node's local neighborhood, allowing predictions to depend on both node attributes and the structure of connections.

GNNs are built from stacked message-passing layers. At each layer k, a node v updates its representation by aggregating information from its neighbors:

$$\mathbf{x}_v^{(k)} = \phi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \mathbf{x}_u^{(k-1)}\right)\right),$$

where $\psi^{(k)}$ is a function aggregating a node's neighborhood, and $\phi^{(k)}$ is an update function combining the node's own features with the aggregated neighborhood information. The aggregation function is typically chosen to be permutation-invariant with respect to the ordering of neighboring nodes, and the update function is usually the learnable component.

After K layers, the final node representation encodes information not only about the node itself but also about its K-hop neighborhood. This iterative message passing enables the GNN to capture long-range dependencies between K-hop neighbors, which are inaccessible to standard neural networks.

The key inductive bias of GNNs is therefore relational: predictions are not made in isolation but are conditioned on the local graph structure. While this increases computational complexity and introduces challenges for scaling, it allows the model to exploit information that would otherwise be lost under the *i.i.d.* assumption.

2.4 GRAPH NEURAL NETWORKS AND LARGE GRAPHS

- Single layer $\rightarrow$ unbiased estimator of mean. Multiple layers $\rightarrow$ nested sampling $\rightarrow$ nonlinear composition $\rightarrow$ biased.
- GraphSAGE does not correct for unequal sampling probabilities or degree-dependent sampling imbalance.

- GNNs often have few layers; that is why they work well in practice even though the sampling may be biased.

In typical neural networks the datapoints are either completely independent, or in some cases dependent but with a structure that makes it straightforward to determine which datapoints depend on which.

GNNs lack any such relational inductive bias because there are no architectural assumptions about the dependency structure. Other neural architectures that exhibit inter-datapoint dependencies usually have some built-in relational assumption. Time-series models assume a sequential ordering between datapoints, and convolutional neural networks (CNNs) assume spatial proximity between pixels. These assumptions make it straightforward to identify the set of datapoints on which a given datapoint depends.

In a time-series model one can easily derive that datapoint t_n depends on all $\{t_{n-1}, t_{n-2}, \ldots, t_{n-k}\}$. In a CNN, pixel $p_{x,y}$ depends only on

$$\{\, p_{i,j} \mid |i-x| \leqslant r,\ |j-y| \leqslant r \,\},$$

where r is the receptive-field radius, and pixels are never dependent on pixels in other images.

We say that x_1 depends on x_2 if:

$$f(x_1) \neq f(x_1 \mid x_2) \quad \text{or} \quad f(x_1) \neq f(x_1, x_2). \tag{1}$$

There are different degrees of strictness in such dependencies. Time-series models often maintain an internal state summarizing past observations: datapoints only need to be presented in the correct order. To make an accurate prediction at timestep t one merely needs to have previously inserted $x_0, x_1, \ldots, x_{t-1}$ and then insert x_t at the current step (the relaxed form in the left-hand side of Equation 1). CNNs and GNNs are stricter: all datapoints that x depends on must be provided as immediate inputs to the model.

2.4.1 *Training on Large Datasets Under Memory Constraints*

Training on a dataset of X GB on a machine with Y GB of memory where $Y < X$ is straightforward for typical neural networks with SGD, because it suffices to load datapoints in mini-batches $\{d^{(1)}, d^{(2)}, \ldots, d^{(n)}\}$ such that $\sum_i d^{(i)} < Y$. For weakly structured dependencies one additionally needs to load datapoints in the correct sequence; for immediate structural dependencies (as in CNNs) one must ensure correct batch partitioning.

The problem is compounded for GNNs. The output for node x depends on its K-hop neighborhood. If the average node degree is n,

then the number of nodes each node directly depends on is approximately n^K.

Returning to the social network fraud-detection example: predicting whether a person will make a fraudulent transaction requires, for a 2-layer GNN with an average of 50 transactions per account, approximately $50 \times 50 = 2{,}500$ times more datapoints than a traditional neural network that uses only that person's features.

2.4.2 *Subgraph Sampling*

The node-to-node dependency means that after sampling a mini-batch of seed nodes, one must build a subgraph containing the seed nodes and their neighbors.

The K-hop neighborhood of seed nodes can vary considerably in size. To avoid exploding neighborhood sizes, a sampling algorithm is typically used to select a subset of the true K-hop neighborhood. Sampling strategies vary in implementation, but they all impose an upper bound on the number of nodes sampled at each hop. These bounds are usually specified as $[k_0, k_1, \ldots, k_n]$, where k_i is the maximum number of neighbors to sample at hop i from the seed nodes.

This uniform neighbor sampling renders the mini-batch gradient biased, and therefore does not guarantee convergence for SGD. In practice, however, convergence is generally sufficient:

$$\mathbb{E}_S\left[\nabla_W \ell\left(\sigma\left(\frac{1}{|\mathcal{N}^S(v)|}\sum_{u\in\mathcal{N}^S(v)} h_u^{k-1}\right)\right)\right] \neq \nabla_W \ell\left(\sigma\left(\frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} h_u^{k-1}\right)\right),$$

where the left-hand side is the expected mini-batch gradient over sampled neighbor graphs, and the right-hand side is the full gradient over all neighbors.

Previous work has addressed this problem by partitioning the graph across several worker machines that each train the GNN using the gradient computed from their partition. Partitions are constructed so that clusters of nodes with many common neighbors reside on the same machine, thereby minimizing inter-worker communication.

The paper "Scalable and Efficient Full-Graph GNN Training for Large Graphs" implements a system along these lines with the graph partitioned among k workers. Each worker iteratively generates the output for each message-passing layer, synchronized with the other workers; after each layer it both sends its computed hidden states to workers that require them and receives hidden states for nodes stored elsewhere.

Similarly, the paper "ByteGNN: Efficient Graph Neural Network Training at Large Scale" partitions the graph among a number of workers. The high-level architecture consists of one `GraphStore` responsible for partitioning, paired with sampling and training workers. The sampling workers draw a subgraph around the seed nodes

in the mini-batch and fetch required neighboring nodes from other workers.

Finally, Neo4j has presented a paper "Graph Neural Networks on Graph Databases" demonstrating that it is possible to train a GNN by storing the graph in a Neo4j graph database and sampling subgraphs around the seed nodes for each mini-batch. This is, to the best of our knowledge, the only scientific paper exploring a graph-database solution to the problem of scaling graph neural networks.

2.5 PYTORCH GEOMETRIC

PyTorch Geometric is a graph neural network library that enables easy creation of GNNs in Python. It is built on top of PyTorch and provides helpful modules such as predefined message-passing layers (graph attention, graph convolution, and more), making it straightforward to construct customizable GNNs by combining different layers within implementations of the abstract PyTorch `nn.Module` class.

PyTorch Geometric also provides a framework for storing the graph remotely in a database, through the following abstract classes:

- `FeatureStore`: Defines how node feature vectors are stored and fetched. A fast feature-vector look-up is essential.
- `GraphStore`: Defines how the graph topology is stored. In contrast to the `FeatureStore`, the emphasis here is on fast traversal rather than fast look-up, since efficient neighbor sampling requires rapid graph traversal.
- `BaseSampler`: Allows the user to define a sampling method for selecting neighbors used in the forward pass. The `BaseSampler` is tightly coupled with the `GraphStore`. Multiple sampling processes can be spawned; since instances of the sampler are recreated in each separate process, it is important not to store large amounts of data in the sampler.
- `NodeLoader`: Takes implementations of `FeatureStore`, `GraphStore`, and `BaseSampler` together with batching hyperparameters (e. g., batch size) and provides a data loader that splits the data into mini-batches, samples a subgraph for each batch, and fetches the corresponding node feature vectors from the `FeatureStore`.

2.6 NEO4J

3 METHODOLOGY

A

APPENDIX

DECLARATION

Put your declaration here.

Put location here, February 2026

Harry Wik & Victor Pekkari

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede. The style was inspired by Robert Bringhurst's seminal book on typography "*The Elements of Typographic Style*". `classicthesis` is available for both LaTeX and LyX:

https://bitbucket.org/amiede/classicthesis/

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

http://postcards.miede.de/